

COMPSCI 701:
Creating Maintainable Software

Lecture 6.2: Support for comprehending programs

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Agenda

- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

- Admin
 - Questions?
- L5.3
- PMD & Assignment 1
- Support for comprehending programs
- (No class exercise)

Previously in COMPSCI701 — PMD

- Agenda
- **Previously**
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

- a **linter** is a tool that detects aspects of code believed to lead to future (often maintainability) problems
- PMD is one such linter
- PMD can be customised as to what aspects it checks
- A “ruleset” **pmd.xml** has been provided in the Assignment 1 repo
- This ruleset will be used for the marking of Assignment 1
- Instructions for how you can use PMD yourself are available in the repo.
- Linters (and PMD) will be a topic for later discussion

- Agenda
- Previously
- **PMD in A1**
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

PMD use in Assignment 1

- Number of submissions: 97
- Number with zero violations: 40
- Average overall (/97): 2.4
- Average when had violation (/57) :4.1
- Rules violated (top 11)

Rule	# submissions	# violations
NcssCount	41	85
CouplingBetweenObjects	32	49
CyclomaticComplexity	31	46
ControlStatementBraces	20	85
UnusedPrivateField	13	31
MethodReturnsInternalArray	12	19
AvoidReassigningParameters	8	12
fieldvisibility	8	27
DataClass	8	11
UnusedAssignment	7	19
UnnecessaryImport	7	10

- To be discussed

- Agenda
- Previously
- **PMD in A1**
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

NcssCount

- “This rule uses the NCSS (Non-Commenting Source Statements) metric to determine the number of lines of code in a class, method or constructor. NCSS ignores comments, blank lines, and only counts actual statements.”
- **Thresholds: Class — 75, Method — 25**
- **“Functions/Classes should be small” (for some definition of “small”)**

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

ControlStatementBraces

- “Enforce a policy for braces on control statements. It is recommended to use braces on ‘if ... else’ statements and loop statements, even if they are optional. This usually makes the code clearer, and helps prepare the future when you need to add another statement.”
- E.g.

```
public void isTrue(boolean b) {  
    if (b) System.out.println("true!");  
}
```

Or

```
public void isTrue(boolean b) {  
    if (b)  
        System.out.println("true!");  
}
```

- When expecting braces, if they are not there it can be confusing

fieldVisibility

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

- Fields should not be accessible from other classes (exception, occasionally, ancestor classes)
- “Package private” or “default package” visibility

```
public class MyClass {  
    int myField; // accessible by all classes in same package  
}
```

- Most submissions had all the classes in the same package
- See **Encapsulation**

How to comprehend programs?

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

- The most wonderfully formatted code, with the absolute best names in the world, is not going to help much with a 1-million (or even 10-thousand) line program!
- General approach—create tools that provide some view of the code that focuses on the “important” aspects of the code
- Specific (selected) approaches:
 - Program Analysis Tools
 - Software Visualisation Tools
 - Programming Languages

Program Analysis

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

- Tools that support understanding of code need to analyse the code to determine the properties of interest
- Requires the tool to read (“parse”) the code and create an internal representation of it (typically an “abstract syntax tree”—AST)
- Then the internal representation is examined for features of interest
- Often provide **measurements** from “software metrics”
- Linters do program analysis, although the goal is to identify potential quality issues rather than directly help the comprehension process
- Examples (Honours projects):
 - How do smells change over time?
 - How do we detect “bad smells” in code?
 - How often do developers ignore guidelines on how to name identifiers?
 - How often do developers use names that are not meaningful?
 - How many parameters do developers have in the functions/methods they create?
 - How often do developers record in their code that they have made poor decisions? (Self-Admitted Technical Debt)
 - Is “lines of code” providing different information to “number of methods”, or “number of fields”, or ...
 - How do we determine the degree of “separation of concerns”?

Software Visualisation

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- **Software Visualisation**
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

- A big area of research in software engineering (e.g. see the VISSOFT—the IEEE Working Conference on Software Visualization <https://vissoft.info>)
- Basic idea is “a picture paints a thousand words” — so use pictures to efficiently show aspects of interest in software
- Definition: “The use of the crafts of typography, graphic design, animation, and cinematography with modern human-computer interaction and computer graphics technology to facilitate both the human understanding and effective use of computer software”
(from J. T. Stasko, J. Domingue, M. H. Brown, and B. A. Price, editors. **Software Visualization**. The MIT Press, 1998.)
- Typically **comprehension task specific**

What to visualise?

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

- Most common — measurements from software metrics (e.g. LOC, Number of classes)
- Structure — relationships between elements (typically classes)

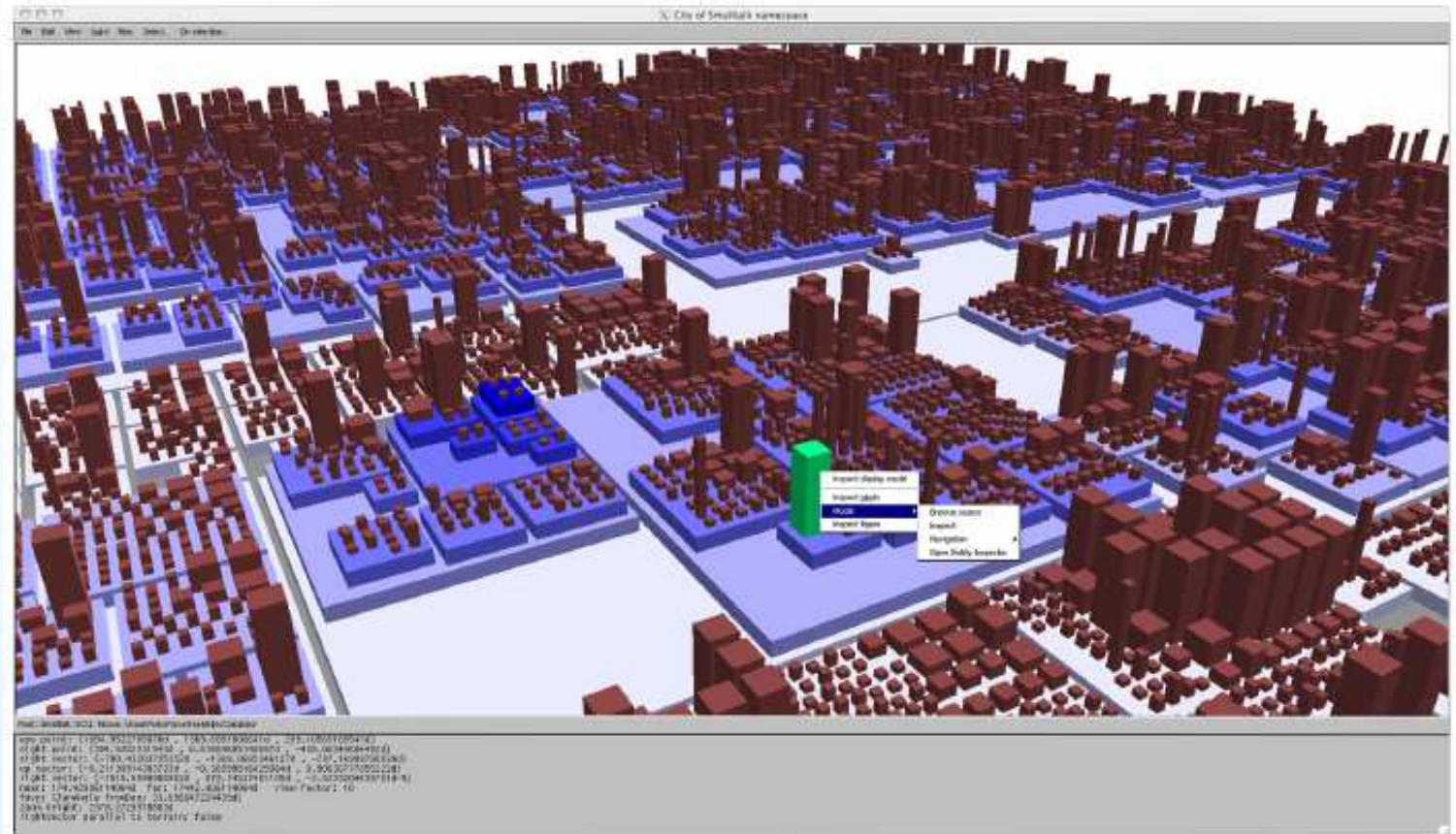
City Metaphor

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- **City Metaphor**
- Object Counts
- ACE
- Remix
- Key Points

- An idea that has been widely adopted to visualise large systems
- Basic idea: represent a software systems as a city, where buildings are classes, districts are packages, size, colour, and other attributes of buildings represent measurements
- Introduced by : R. Wettel and M. Lanza, "Program Comprehension through Software Habitability," *15th IEEE International Conference on Program Comprehension (ICPC '07)*, 2007
- Some research suggests this idea helps with comprehension.
 - 41 participants
 - half used the visualisation and half use an IDE.
 - those using the visualisation showed improvement in effectiveness and efficiency
 - R. Wettel, M. Lanza, R Robbes "Software Systems as Cities: A Controlled Experiment" *International Conference on Software Engineering (ICSE'11)* 2011

Example 1

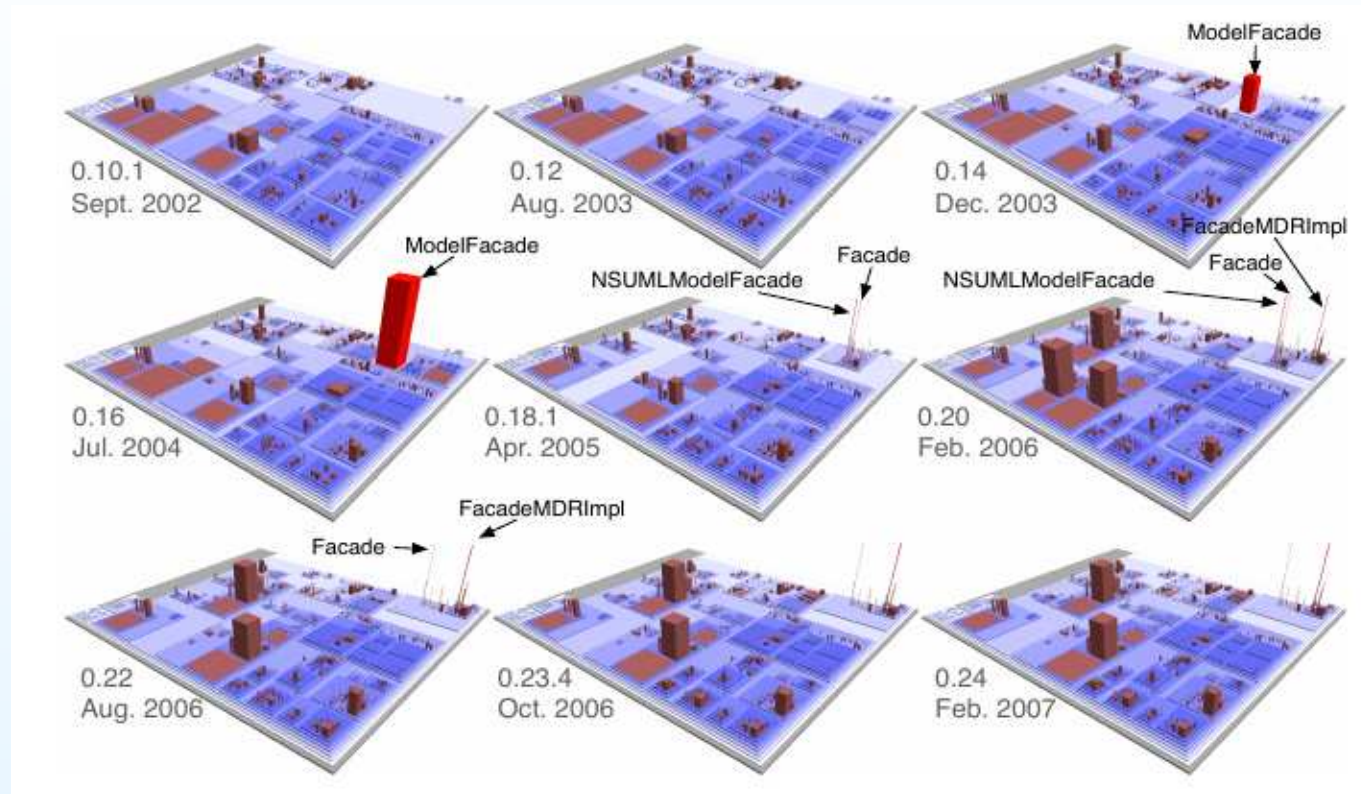
- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- **City Metaphor**
- Object Counts
- ACE
- Remix
- Key Points



- Figure 8 from: R. Wettel and M. Lanza, "Visualizing Software Systems as Cities," *4th IEEE International Workshop on Visualizing Software for Understanding and Analysis* 2007

Example 2: Showing Evolution

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points



- Figure 3 from: R. Wettel and M. Lanza, "Visual Exploration of Large-Scale System Evolution" *15th Working Conference on Reverse Engineering* 2008.

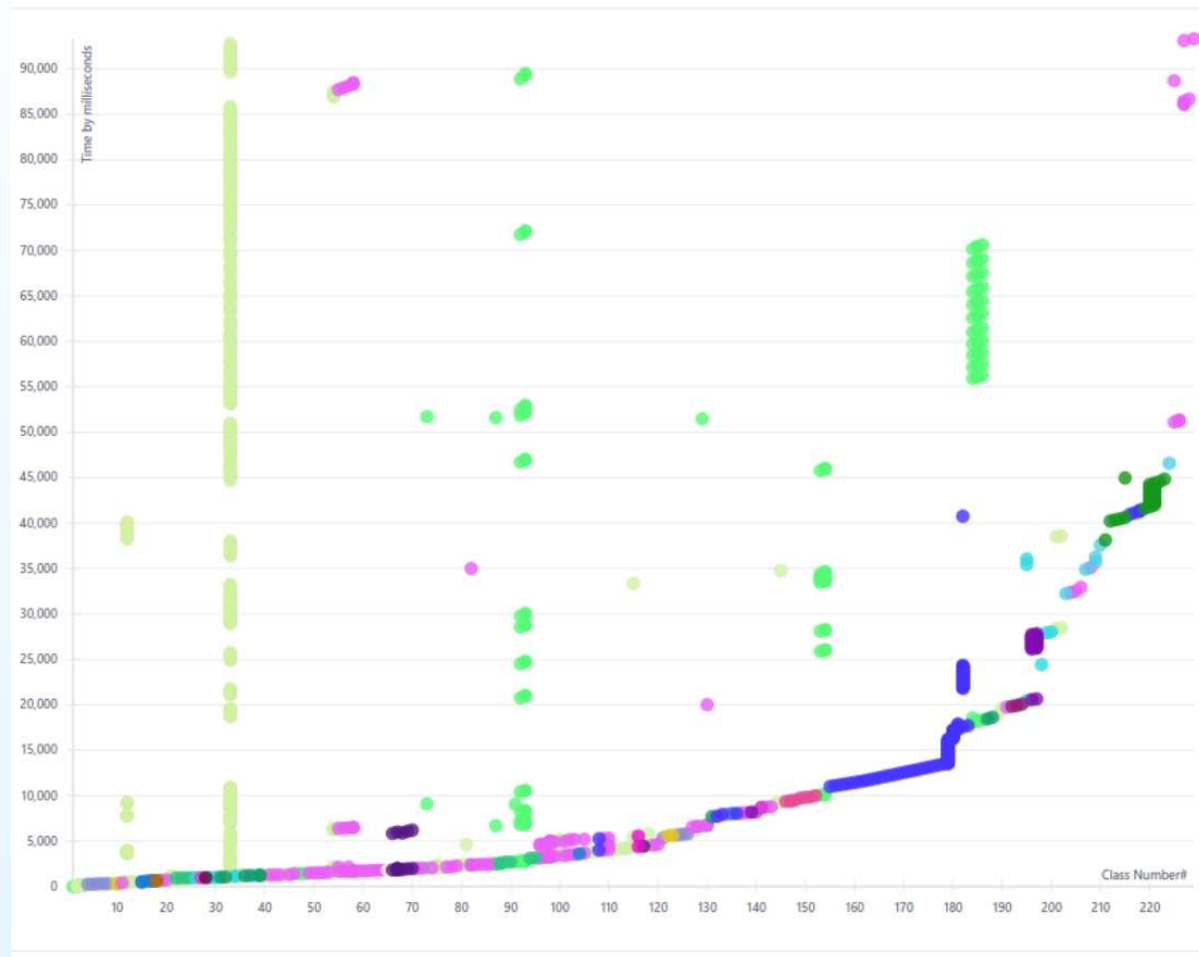
- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

Visualisations of object creation

- Zhihui Wang “Thesis Title: Understanding Java Programs by Counting Objects” M.Sc 2020
- Motivation: Number of objects created helps determine to what degree a class matches the context schema concept it is meant to represent
 - ⇒ Understanding when objects are created may help with program comprehension
- Build a tool that visualises **what** objects are created **when**
- Example: Lucene.
 - Open-source Java application for supporting efficient searching of text files.
 - Basic operations are **indexing** the files and then supporting **searching** of those files
 - About 720,400 LOC

Example: Indexing

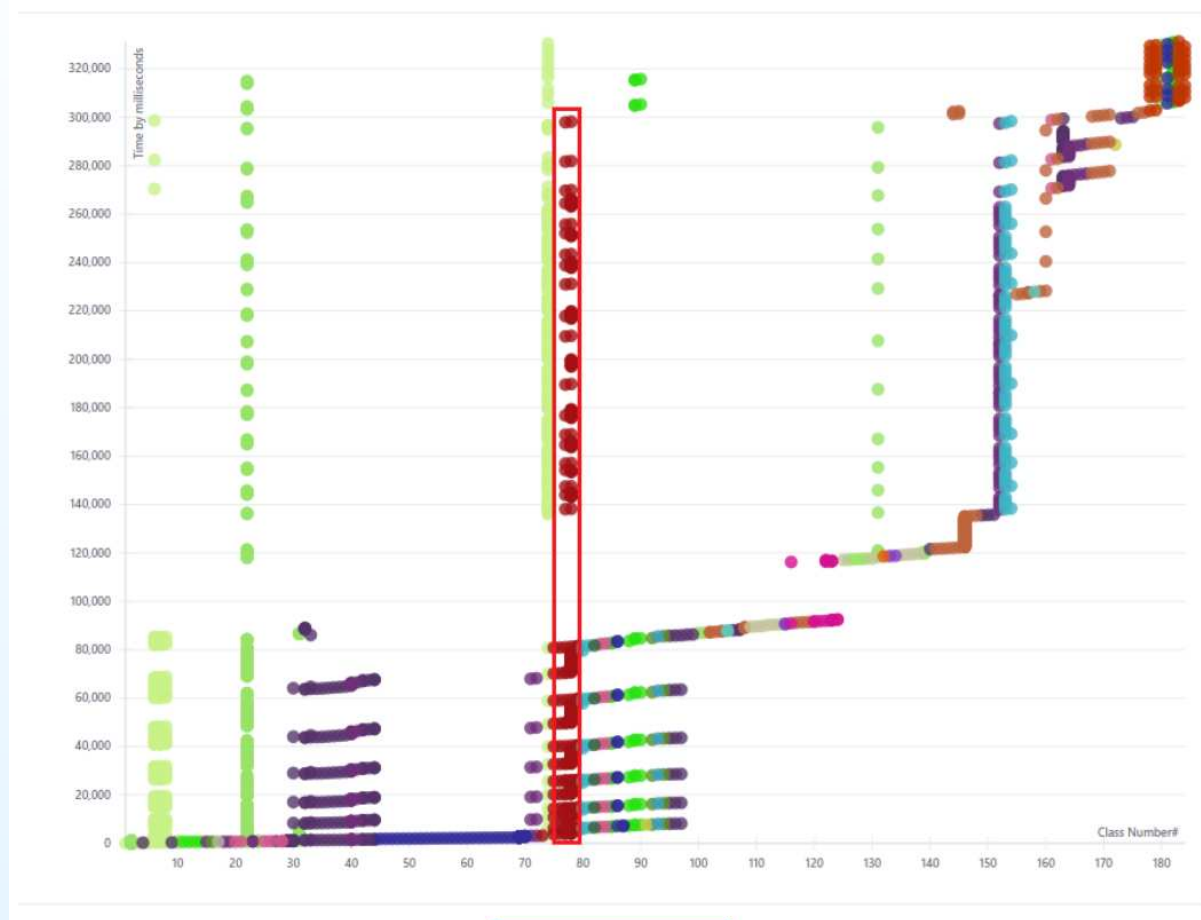
- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points



- y-axis — time (ms), x-axis — object ID, colour — package

Example: Searching

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- **Object Counts**
- ACE
- Remix
- Key Points



- y-axis — time (ms), x-axis — object ID, colour — package

Automated “Concept” Extraction (ACE)

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

- Brand-new (still in development!) work by Hangzhou Fang, Yuanfang Cai, Rick Kazman
- Hypothesis: When trying to understanding a large amount of code, developers look for things the recognise, such as:
 - Design (and Architecture) Patterns — common structure organisations
 - Abstractions — high-level ideas (typically from problem)
 - Across Abstractions — characteristics that occur across abstractions
 - Across Packages — characteristics that occur across packages
- Assumption: Developers use meaningful names in a consistent manner **Threat to Validity**
- Seeing dependencies that exist between these different “concepts” help with understanding

H. Fang et al., “A Holistic Approach to Design Understanding Through Concept Explanation,” in IEEE Transactions on Software Engineering, vol. 51, no. 2, pp. 449-465, Feb. 2025, doi: 10.1109/TSE.2024.3522973

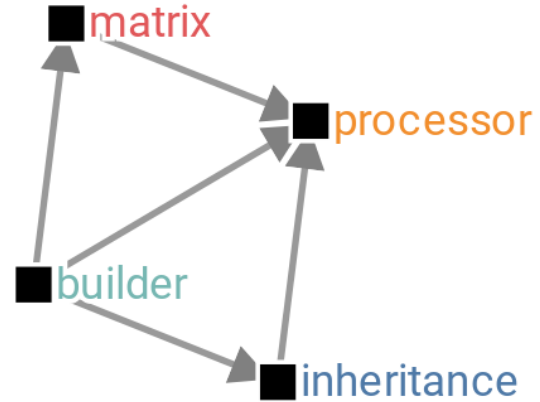
- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- **ACE**
- Remix
- Key Points

Extraction Process

- Divide (“split”) names into component “keywords”
`InheritanceMatrixProcessorBuilder` becomes **Inheritance**, **Matrix**, **Processor**, and **Builder**
- Look for classes (or interfaces) at the top of inheritance hierarchies corresponding to keywords
`Inheritance`, `Processor`, `Matrix`
- Look for keywords appearing in multiple places
`FilenameProcessorBuilder` and `InheritanceMatrixProcessorBuilder`
- Look for keywords corresponding to patterns (which, by definition, are known)
`FilenameProcessorBuilder` and `InheritanceMatrixProcessorBuilder`
- Some keywords indicate multiple concepts

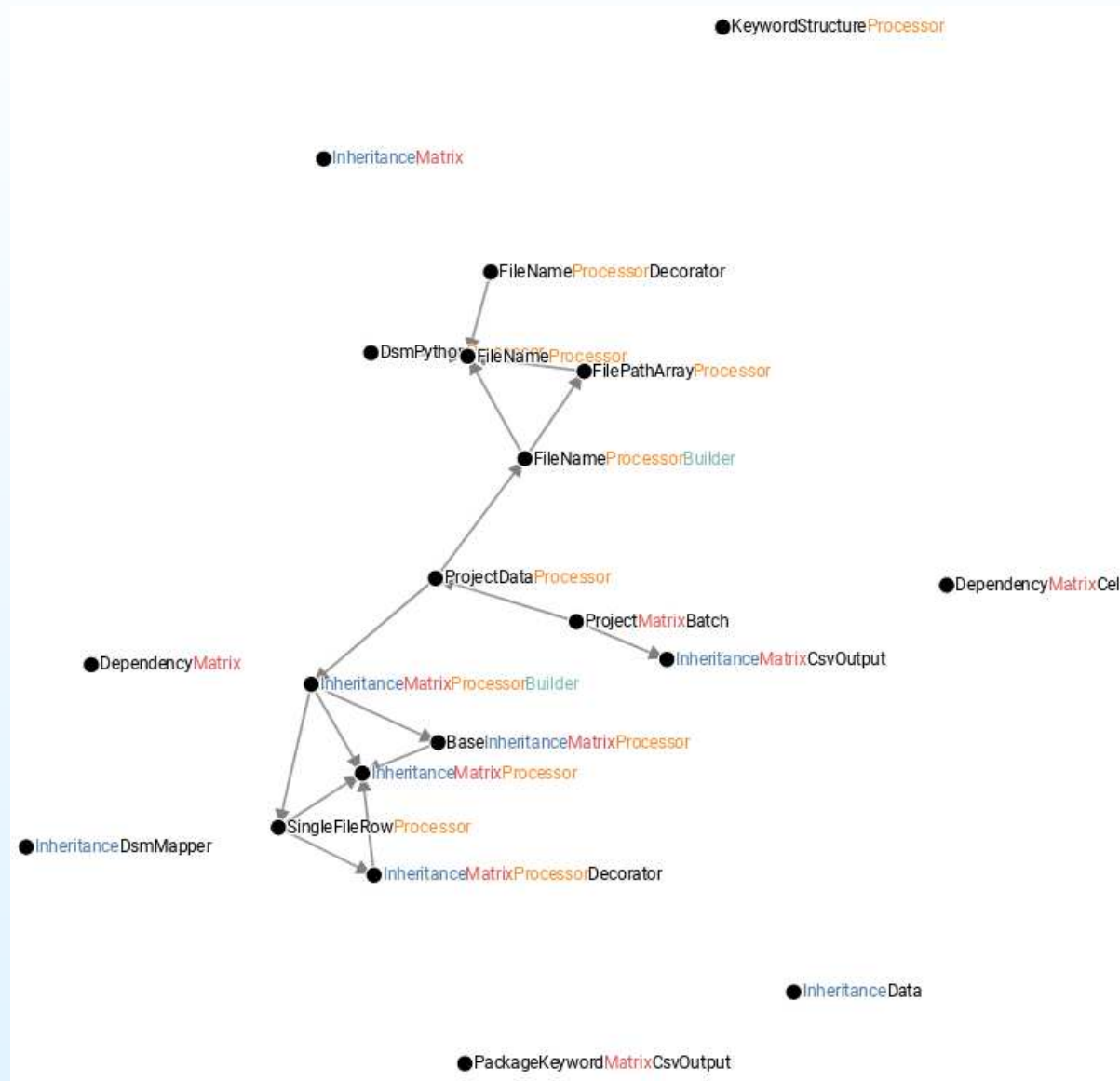
ACE Example 1

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- **ACE**
- Remix
- Key Points



ACE Example 2

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points



- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- **ACE**
- Remix
- Key Points

Unified Modelling Language (UML)

- understanding an object-oriented design usually focuses on “classes” and their relationships, with methods and fields sometimes but often not important (and implementations not at all important)
- $\Rightarrow$ only show classes and their relationships! (and method some field and method information)
- Such information can be shown by drawing shapes and connecting them with lines
- Different shapes can be used to indicate some information (e.g. class versus interface) and lines can be annotated to show different information (e.g type of arrow to show composition versus inheritance)
- People started formalising the notations used as to what they all meant
- But different people use different shapes and different annotations for the same thing, in particular Jim Rumbaugh created OMT, Ivar Jacobson OOSE, and Grady Booch created the Booch method, but all different
- Rumbaugh, Jacobson, and Booch (the “Three Amigos”) weren’t the only ones with formal notations, but they got together to “unify” their different techniques
- Hence “Unified Modelling Language”

UML

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

- UML is a visual language that can be used to describe software
- UML shows different views of software, e.g. static (class diagrams), dynamic (sequence diagrams), from the user view (use case diagrams), the relationship between software and hardware (deployment diagrams), and so on
- UML provides a visualisation of software, but does it aid comprehensibility?

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- **Remix**
- Key Points

The Remix Programming Language

- Consider the following:

"The Java experts are: " the "Java" experts amongst the (students)

Imagine writing code like and have it mean something?

- The declaration of the “experts amongst the” function

```
the (language) experts amongst the (group) :  
  experts : {}  
  for each (programmer) in (group) do  
    if ((programmer) is an expert in (language))  
      append (programmer's name) to (experts)  
  experts
```

- The declaration of “students” is

```
students : {  
  "Martin Developer" age 22 knows {  
    Java : "Expert"  
    C : "Expert"  
    python : "Advanced"  
    Remix : "Expert"  
  }  
  ...  
}
```


- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

The Remix Programming Language

- Remix is a programming language developed by Robert Sheehan (retired) with the goal to be readable
- Remix uses standard compiler technology (in particular ANTLR, a Java parser-generator) — there is nothing particularly new so no reason why we don't already have languages like this
- Does this solve all of our problems for writing comprehensible code?

- Agenda
- Previously
- PMD in A1
- Motivation
- Program Analysis
- Software Visualisation
- What to visualise?
- City Metaphor
- Object Counts
- ACE
- Remix
- Key Points

Key Points

- There are means to support understanding code that does not necessarily require the code to be comprehensible
- But the more comprehensible it is, the better the support is likely to be
- Limited evidence that software visualisation helps
- Perhaps the need for such support is an admission we write code that's hard to understand!